

# CS 331: Computer Networks

## Scalable Network I/O: From Select to io\_uring

### Project Report

#### 🌀 Introduction:

Internet is a Network of Networks. Millions of clients may try to connect to servers simultaneously across the Internet. In such a scenario, it becomes crucial for a server to handle a large number of client connections efficiently.

To understand the problem, let us first consider the basic TCP client-server model. A server typically follows these steps:

1. **Socket Creation:** The Server first creates a socket using the `socket()` system call.
2. **Binding:** The server binds this socket to a specific local IP address and port number via `bind()`.
3. **Listening:** The server then starts listening for incoming connection requests. At this stage, the socket is referred to as a passive **Listening Socket**. It can now form a queue of incoming connection requests.
4. **Accepting:** When a client attempts to connect to the server, the server accepts the connection. The `accept()` operation returns a new socket, called a **Connection Socket**, which is used to communicate with that client.

#### ➔ The Challenge of Scalability:

Now consider this listening server has 10,000 connected clients. The server needs an efficient way to determine which of these clients currently require attention, such as having data available to read. If the server repeatedly checks every connected client, a significant amount of time and processing may be spent checking sockets that are not ready.

While this approach may work for a smaller number of connections, but then as the number of simultaneous connections increases further, for example to hundreds of thousands or even millions of clients, this approach becomes increasingly inefficient and creates scalability challenges. This leads us to the need for better and more efficient mechanisms for monitoring and handling Network I/O.

And this is what this project is for! We will explore the evolution of Network I/O mechanisms from `select()` and `poll()` to `epoll` and `io_uring` and then compare their approaches to handling large numbers of concurrent network connections.

Let's meet our team who worked on this project and then move to exploring the Network IO History:

#### 🌀 Team Members:

| Sr. No. | Name                      | Roll No. | Contributions |
|---------|---------------------------|----------|---------------|
| 1.      | Chitiveli Hemcharan Varma | 24110095 | Benchmarking  |
| 2.      | Gella Naga Sai Krishna    | 24110122 | io_uring      |
| 3.      | Guransh Kaur Saran        | 24110127 | Select        |
| 4.      | Kowel P Laloo             | 24110168 | Poll          |
| 5.      | Pallav Biyala             | 24110234 | Epoll         |

## 🌀 Fundamental I/O Modes:

Before going into the history of Network and seeing how specific multiplexing mechanisms work, it is important for us to know how Linux kernel handles socket read/write operations under different I/O modes.

So, let's see two important modes:

### → Blocking I/O Mode:

Suppose the connection socket *S* is assigned to some client *A*. The server now wants to read data from this client socket.

However, it is possible that client *A* has not sent any data yet. In this case, when the server performs a blocking `read()` operation, it waits for data to become available. The thread performing the operation is put into a **waiting state** by the operating system and is woken up when data becomes available or an error occurs.

In a simple single-threaded server, if the server is blocked waiting for data from client *A*, it cannot proceed to handle other clients during this time. Even if other clients have data ready to be processed, the server remains occupied with the first client.

This waiting behavior is why the operation is called **Blocking I/O**.

Clearly, using this approach for a large number of clients can lead to significant delays. If the server has to wait for each client individually before moving on to the next one, a large amount of time can be spent waiting rather than performing useful work.

This leads us to the need for:

### → Non-Blocking I/O Mode:

Instead of forcing the server to pause and wait for an idle client, **non-blocking I/O** adopts a different approach:

*"If the current client does not have data, that's fine! Let's attend the other clients."*

In Non-Blocking Mode, the server does not wait for a particular client to send data.

#### Note:

By configuring client and server sockets with the `O_NONBLOCK` flag (using the `fcntl()` system call), the Non-Blocking Mode can be turned on.

When it attempts to read from a client socket, the `read()` operation returns immediately if data is available. If no data is available, instead of putting the server thread into a waiting state, the operation returns without blocking, allowing the server to continue handling other clients.

More technically, if a server attempts to read from a socket that has no pending data in its buffer, the Operating System instantly returns error codes: `EAGAIN` or `EWOULDBLOCK` ("resource temporarily unavailable") and hence then server can continue handling other clients.

This solves the problem of waiting for each client individually and can significantly reduce unnecessary waiting. The server can therefore make progress on other connections instead of being stuck waiting for one client. This reduces the response latency and allows a single thread to loop through multiple client sockets.

## ☞ The Evolution of the Network I/O:

As said by **George Bernard Shaw**:

*Science never solves a problem without creating ten more*

So we solved the problem of blocking, but created a new one:

**If the server has thousands of client sockets, how does it efficiently determine which sockets are actually ready for I/O?**

This means now server has to choose which client sockets are ready and need attention. If it went to a client socket and found out, it isn't ready with data, it would waste cycles.

And this leads to what our whole project is all about: **I/O Multiplexing**.

Let's start with how the whole evolution of I/O Multiplexing happened, starting with the first method introduced in 1983:

→ **select Multiplexing:**

Now that we understand why non-blocking I/O alone is not enough, let us look at the first solution Linux provided to the "which sockets are ready?" problem: the `select()` system call, introduced in 1983.

Δ **What is select?**

`select()` is a system call that allows a server to monitor multiple file descriptors simultaneously and determine which of them are ready for reading, writing, or have encountered an error, all in a single blocking call.

Instead of the server manually polling every client socket in a loop, it hands the entire list of client sockets to the kernel and says:

*"Here are all the sockets I care about (active lists). Wake me up when any of them are ready."*

The kernel then checks all those sockets and returns control to the server only when at least one of them has something to do.

† **The Example of a Teacher Taking Attendance:**

Imagine a teacher has a classroom of, say, 1000 students. The teacher needs to know which students have completed their assignment and are ready to submit it.

The most straightforward approach, which is essentially what `select` does, is to perform a roll call:

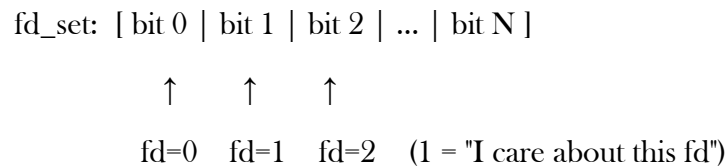
- The teacher reads out every student's name from a list, one by one.
- Each student answers "Ready!" or stays silent.
- After going through the entire list, the teacher now knows which students are ready.
- The teacher then attends to those students.

But here is the catch: the teacher must read out every single name every single time, even if only three students are ready and the rest have nothing to submit. As the class size grows, the roll call becomes longer and longer, even if almost no one is ready.

This is exactly how select works. It is thorough, it is correct, but it is expensive when the number of monitored sockets grows large.

### Δ How does select work?

The key data structure in select is the `fd_set`, a fixed-size bitmap where each bit represents one file descriptor.



The server builds this bitmap before every call to `select()`, marking the bits corresponding to the sockets it wants to monitor. It then hands this bitmap to the kernel.

There are three important operations involved:

#### 1. `FD_ZERO()` and `FD_SET()` - Building the watch list:

Before each call to `select()`, the server must:

- Clear the entire bitmap using `FD_ZERO()`.
- Then mark every active client socket's bit using `FD_SET(fd, &readfds)`.

This rebuild happens from scratch on every single iteration of the event loop, every time the server wants to check for new events. This is one of the key inefficiencies of select: the kernel has no memory of what it monitored last time.

#### 2. `select()` - The system call:

```
int activity = select(max_fd + 1, &readfds, &writefds, NULL, NULL);
```

The server passes:

- `max_fd + 1`: the range of file descriptors to scan (the kernel checks fds 0 through `max_fd`).
- `&readfds`: the set of fds to watch for incoming data (reading readiness).
- `&writefds`: the set of fds to watch for send-buffer availability (writing readiness).
- `NULL, NULL`: no exception set, no timeout (block forever until something is ready).

The kernel then scans every bit in the bitmap, checks each corresponding socket's state, and marks the bits of sockets that are ready. It copies this modified bitmap back to user space.

The server then uses `FD_ISSET(fd, &readfds)` to check, bit by bit, which sockets became ready.

This means two  $O(N)$  passes occur on every event:

1. The server rebuilds the full bitmap -  $O(N)$ .
2. The kernel copies it in, scans it, copies it back -  $O(N)$ .
3. The server scans all  $N$  bits with `FD_ISSET` to find the ready ones -  $O(N)$ .

### 3. `FD_ISSET()` – Checking the results:

```
if (FD_ISSET(client_fd, &readfds)) {
    // this client has data ready to read
    recv(client_fd, buf, BUFFER_SIZE, 0);
}
```

For every socket in the client list, the server checks whether its bit was set in the returned bitmap. If yes, data is ready and the server can call `recv()` without blocking.

### Δ The Hard Limit Problem with `FD_SETSIZE`:

`select` has a critical structural limitation: the `fd_set` bitmap is a fixed-size array defined at compile time by the constant `FD_SETSIZE`, which is 1024 on Linux by default.

This means:

- `select` can only monitor file descriptors numbered 0 through 1023.
- If a client is assigned a file descriptor number  $\geq 1024$ , calling `FD_SET(fd, &readfds)` with that `fd` silently overflows the bitmap into adjacent memory which corrupts data.

This is not a soft limit that can be raised at runtime. It is a memory layout constraint baked into glibc's `fd_set` type.

For a server that needs to handle thousands of simultaneous connections, this makes the standard `select` completely unusable without modification.

### Δ Our Fix - Custom `my_fd_set`:

To benchmark `select` fairly alongside the other engines at up to 5000 connections, we implemented a custom bitmap struct that bypasses the `FD_SETSIZE` restriction:

```
#define MAX_CLIENTS 10000
#define OUT_BUF_CAP 65536

// Custom fd_set definition to safely bypass glibc's hardcoded FD_SETSIZE=1024 limit and FORTIFY_SOURCE
#define MY_FD_SETSIZE MAX_CLIENTS
typedef struct {
    long int fds_bits[MY_FD_SETSIZE / (8 * sizeof(long int)) + 1];
} my_fd_set;

#define MY_FD_SET(fd, set) (((set)->fds_bits[(fd) / (8 * sizeof(long))]) |= (1UL << ((fd) % (8 * sizeof(long)))))
#define MY_FD_ISSET(fd, set) (((set)->fds_bits[(fd) / (8 * sizeof(long))]) & (1UL << ((fd) % (8 * sizeof(long))))) != 0
#define MY_FD_ZERO(set) memset((set), 0, sizeof(*(set)))
```

This is the same bit-manipulation logic glibc uses internally, just without the `FD_SETSIZE` guard. Importantly, `MY_FD_SETSIZE` is set equal to `MAX_CLIENTS = 1024`, the **bitmap is the same size as the standard `fd_set`**, not larger.

**We are not attempting** to extend `select` beyond 1024 fds, because the Linux kernel makes that impossible regardless.

When calling the actual `select()` system call, we cast our `my_fd_set*` to `fd_set*`:

```
int activity = select(max_fd + 1, (fd_set*)&readfds, (fd_set*)&writefds, NULL, NULL);
```

This is valid because the kernel does not inspect the C type, it only reads raw memory at the address provided, using the `max_fd + 1` argument to know how far to scan. As long as `max_fd` stays below 1024, this is safe and correct.

### → Why we accept the 1024 limit:

Because `select`'s 1024-fd ceiling is a kernel-enforced constraint, not a userspace one. Attempting to work around it with a larger bitmap would only cause the kernel to silently ignore the extra bits, producing incorrect results. Our implementation honestly enforces this limit: any connection assigned an `fd ≥ MY_FD_SETSIZE` is explicitly rejected:

```
} else if (new_fd >= MY_FD_SETSIZE) {  
    printf("Rejecting fd=%d, exceeds MY_FD_SETSIZE\n", new_fd);  
    close(new_fd);  
    continue;
```

This means `server_select` is capped at approximately **1000 concurrent connections** in practice, and our benchmark results reflect this: the server's throughput and latency data are valid only for  $N \leq 1000$ ; at  $N = 1500$  and beyond, the server cannot accept connections and the benchmark records NaN.

### Δ Handling Partial Writes using `writefds`:

A naïve `send()` call can silently lose data when the kernel's socket send-buffer is full. `select` supports watching sockets for write readiness via its `writefds` parameter.

Our implementation adds a per-client output buffer (`out_buf[65536]`) and registers a socket in `writefds` only when it has unsent bytes:

```
for(int i=0; i<num_clients; i++){  
    MY_FD_SET(clients[i]->fd, &readfds);  
    if (clients[i]->out_len > 0) {  
        MY_FD_SET(clients[i]->fd, &writefds);  
    }  
    if (clients[i]->fd > max_fd) {  
        max_fd = clients[i]->fd;  
    }  
}
```

This is a key advantage of select's design: because the bitmap is rebuilt from scratch on every call, toggling write-interest for a client requires no extra system call — you simply include or omit that fd from writefds before the next select(). (Contrast this with epoll, where changing interest requires a separate epoll\_ctl(EPOLL\_CTL\_MOD) call.)

When select() returns a socket as write-ready, the server calls flush\_outbound\_buffer(), which loops send() until either all buffered data is sent or EAGAIN is returned:

```
int flush_outbound_buffer(client_state *state) {
    while (state->out_len > 0) {
        ssize_t sent = send(state->fd, state->out_buf, state->out_len, 0);

        if (sent < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) break;
            if (errno == EINTR) continue;
            perror("send failed");
            return -1;
        }
        if (sent > 0) {
            memmove(state->out_buf, state->out_buf + sent, state->out_len - sent);
            state->out_len -= sent;
        }
    }
    return 0;
}
```

Note that unlike the epoll version, flush\_outbound\_buffer here takes only a client\_state\* — it does not need an epoll\_fd argument, because there is no kernel-side interest list to update. The bitmap is simply rebuilt with or without the write bit on the next iteration.

This ensures no data is ever silently dropped under backpressure.

### Δ Our select Implementation:

Here is a step-by-step walkthrough of how our server\_select.c works:

#### Step 1 - Create the listening socket:

Using the shared create\_server\_socket() helper: socket() → setsockopt(SO\_REUSEADDR) → bind() → listen() → fcntl(O\_NONBLOCK).

#### Step 2 - Event loop begins:

On every iteration, the server:

- Calls MY\_FD\_ZERO() on both readfds and writefds to clear the bitmaps.
- Calls MY\_FD\_SET(listen\_fd, &readfds) to always watch for new connections.
- Loops through all active client states, setting their fds in readfds, and in writefds if they have buffered unsent data.

- Tracks `max_fd`, the highest fd number seen, because `select` scans from 0 to `max_fd`.

### Step 3 - Call `select()`:

```
int activity = select(max_fd + 1, (fd_set*)&readfds, (fd_set*)&writefds, NULL, NULL);
```

The server blocks here until at least one socket becomes ready.

### Step 4 - Accept new connections:

If `MY_FD_ISSET(listen_fd, &readfds)` is true, there are pending connection requests. The server drains the accept queue in a `while(1)` loop until `accept()` returns `EAGAIN`, sets each new client socket to non-blocking, applies `TCP_NODELAY`, creates a `client_state` with a 64 KB output buffer, and adds it to the `clients[]` array.

### Step 5 - Handle active clients:

The server loops through all clients:

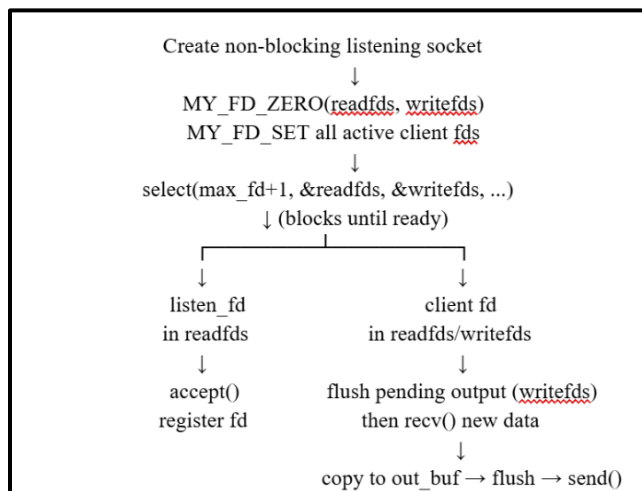
- If `writefds` is set for that fd → call `flush_outbound_buffer()` to drain any pending output.
- If `readfds` is set for that fd → call `recv()`. If data arrives, copy it to `out_buf` and immediately attempt a flush. If `recv()` returns 0, the client has disconnected, so we close and remove.

### Step 6 - O(1) client removal:

When a client disconnects or errors, instead of shifting the entire array left ( $O(N)$ ), the server swaps the departing client's slot with the last active client, then decrements the count. The loop index is decremented by one to re-check the swapped-in client on the next iteration.

```
clients[i] = clients[num_clients - 1];
num_clients--;
i--;
```

A high-level view of this flow is shown below:



### Δ Limitations of `select` :

Even with our my\_fd\_set extension, select carries structural overhead that cannot be optimised away:

| Limitation                                  | Impact                                                          |
|---------------------------------------------|-----------------------------------------------------------------|
| Full bitmap rebuilt every call              | $O(N)$ CPU work per event, regardless of how many fds are ready |
| Full bitmap copied user→kernel→user         | $O(N)$ memory bandwidth per call                                |
| Server must scan all $N$ bits with FD_ISSET | $O(N)$ user-space scan after every select() return              |
| max_fd determines scan range                | Kernel scans fds 0 through max_fd even if most are unused       |

As the number of connections grows, these  $O(N)$  costs accumulate. This led to the development of poll, which removed the 1024-fd limit, and eventually epoll, which eliminated the  $O(N)$  scan entirely.

### → poll Multiplexing:

#### Δ What is poll?

poll() is an I/O multiplexing system call introduced in System V Release 3 (1986) as a direct evolutionary successor to select(). It allows a single-threaded server to monitor multiple socket file descriptors simultaneously and determine which ones are ready for reading (POLLIN) or writing (POLLOUT) in a single blocking call. Instead of using fixed-size bitmasks (fd\_set), poll() uses a dynamically allocated array of struct pollfd structures passed by pointer.

### → The Example of a Waiter Checking Table Cards:

Imagine a waiter managing a dining room with 10,000 customers. Instead of calling out every student's name from a fixed roll-call sheet like select(), the waiter places a reusable request card (struct pollfd) on every table. Customers write on their card whether they want to order (POLLIN) or need the bill (POLLOUT). When checking for orders, the waiter must still physically walk from table 1 to table 10,000 in sequence to inspect each card. Even if only 2 out of 10,000 customers have marked their card, the waiter visits all 10,000 tables on every pass. This permanently breaks the hard 1,024 table ceiling of select(), but the waiter still spends most of their time walking past idle tables.

#### Δ How does poll work?

The core data structure in poll() is struct pollfd, which consolidates state tracking for a single socket into three distinct fields:

```
struct pollfd { int fd; /* File descriptor to monitor (-1 to ignore) */ short events; /* Bitmask of events requested by user (read-only for kernel) */ short revents; /* Bitmask of events returned by kernel (written by kernel) */ };
```

- fd: The integer socket file descriptor. Setting fd = -1 instructs the kernel to ignore the slot during its scan.
- events: The requested interest bitmask set by user space (e.g., POLLIN, POLLOUT). The kernel reads this field but never modifies it.

- revents: The outcome bitmask written exclusively by the kernel upon return (e.g., POLLIN, POLLOUT, POLLERR, POLLHUP, POLLNVAL).

#### Δ State Persistence (No Re-initialization):

Because poll() separates input interests (events) from kernel outcomes (revents), the kernel never overwrites events. Unlike select(), which forces a full bitmap rebuild on every iteration, poll() allows the server to reuse the exact same fds[] array across loop cycles.

#### Δ Breaking the 1024 Ceiling:

Because poll() takes a pointer to an array (struct pollfd \*fds) and an explicit count (nfdst nfdst), the array can be allocated on the heap to monitor 10,000+ connections, completely bypassing select()'s fixed compile-time FD\_SETSIZE limit.

#### Δ Handling Partial Writes using POLLOUT:

All client sockets are configured as non-blocking (O\_NONBLOCK). When a client socket's kernel Transmit (Tx) buffer fills up, send() returns -1 with EAGAIN or EWOULDBLOCK. The server buffers remaining unsent bytes in a per-client output buffer (out\_buf). POLLOUT is added to fds[idx].events only when out\_len > 0. The moment the buffer is completely flushed, POLLOUT is removed (fds[idx].events = POLLIN). This prevents poll() from waking up continuously on writable empty buffers, eliminating 100% CPU busy-waiting.

#### Δ Our poll Implementation (server\_poll.c):

- Step 1 — Create non-blocking listening socket: Using create\_server\_socket() helper: socket() → setsockopt(SO\_REUSEADDR) → bind() → listen() → fcntl(O\_NONBLOCK).
- Step 2 — Initialize parallel arrays: Allocate parallel heap arrays (struct pollfd \*fds and client\_state \*\*clients) sized to MAX\_CLIENTS + 1. Index 0 is assigned to the welcoming socket watching POLLIN.
- Step 3 — Event loop & poll() system call: Invoke poll(fds, nfdst, -1). On every call, the kernel copies all nfdst structures from user space to kernel space, scans them linearly, and copies them back with updated revents.
- Step 4 — Accept new connections: If fds.revents & POLLIN is true, loop accept() until it returns EAGAIN to drain the accept queue. Set each client socket to non-blocking (set\_socket\_non\_blocking), apply TCP\_NODELAY, initialize client\_state, and append to fds[nfdst].
- Step 5 — Handle active clients: Loop through client indices fds[1..nfdst]. If revents & POLLOUT is set, call flush\_outbound\_buffer(). If revents & POLLIN is set, call recv(). Buffer incoming data to out\_buf and attempt an immediate flush.
- Step 6 — O(1) Client Removal (Swap-and-Pop): When a client disconnects (recv() == 0 or error), swap the dead slot fds[i] with the last active client fds[nfdst - 1], decrement nfdst, and decrement i to re-evaluate the swapped element. This avoids an O(N) left-shift memory array move.

#### Δ Limitations of poll:

Despite eliminating select()'s compile-time limit, poll() carries structural overhead that cannot be optimized away:

| Limitation                                  | Impact                                                                                                                                    |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Full Array User → Kernel → User Copy</b> | Copies all nfds structs round-trip across the user-kernel boundary on every single call ( $O(N)$ memory bandwidth tax).                   |
| <b>Linear Kernel Scan</b>                   | Kernel blindly iterates through all nfds sockets checking TCP Rx/Tx buffer states ( $O(N)$ CPU cycles).                                   |
| <b>Linear User-Space Scan</b>               | Application must loop through <code>fds[1..nfds]</code> checking <code>revents != 0</code> after every unblock ( $O(N)$ user-space pass). |
| <b>Quadratic Latency Degradation</b>        | CPU usage spikes to 100% and tail latency degrades quadratically as concurrent connections scale toward 10,000.                           |

As we saw in `select` and `poll`, we had an  $O(n)$  scan of all file descriptors (FDs). Since a server can have many FDs, performing this linear scan repeatedly can become inefficient.

### → **epoll Multiplexing:**

As we saw in `select` and `poll`, we had an  $O(n)$  scan of all file descriptors (FDs). Since a server can have many FDs, performing this linear scan repeatedly can become inefficient.

To address the scalability limitations of `select` and `poll` and improve the handling of large numbers of network connections, Linux introduced a new mechanism in 2002: **epoll**. It provided a more scalable approach to monitoring file descriptors for I/O events.

### Δ **What is epoll?**

Epoll is a Linux kernel I/O event notification mechanism that allows a server to efficiently monitor a large number of file descriptors (FDs) for I/O events.

Let's understand epoll with a very basic example and then go deeper into it:

### † **The Example of a Restaurant:**

Suppose we are a waiter in a restaurant and we have, just say, 10,000 customers to attend. Now the most ordinary way that `select` and `poll` would handle this is:

- Go to each and every table
- Ask them, "Sir, your order ready?"
- If yes, take order and serve them.
- If not go to next customer.

And we would have to repeat this process every time, even if most of the customers do not need our attention. Clearly this wastes a lot of energy of the waiter.

But then the restaurant manager comes up with an idea: **let's have a bell on each table.**

Now, whenever a customer needs something, they simply ring the bell. The waiter does not need to visit every table and ask whether they need anything. Instead, the waiter can continue doing other tasks and attend only those tables where the bell is ringing.

This is exactly what **epoll** does!

### † What epoll do?

Epoll provides a mechanism through which the server can tell the **Linux kernel** which file descriptors it is interested in monitoring. The kernel then monitors these registered file descriptors and notifies the server when one or more of them become ready for an I/O operation.

**For example:** If a client tries to connect to the server, the listening socket becomes ready for reading. The kernel can then notify the server:

*“A connection is waiting on this socket. Please attend to it.”*

The server can then call **accept()** and handle the new connection.

Similarly, if an already-connected client sends data, the corresponding client socket becomes ready for reading, and epoll notifies the server that the socket needs attention.

This means that the server does not have to repeatedly check every client's socket to determine whether it is ready. Instead, it can tell the kernel:

*“Wake me up when one of the sockets I am interested in becomes ready.”*

In this way, epoll avoids repeatedly traversing the entire set of monitored FDs and allows the server to focus its attention on the FDs that are actually ready.

### Δ How the bell gets installed?

Just as the restaurant manager has to do something to buy the bells and install them on the tables, similarly, the first phase of using epoll is to **set up the epoll instance**.

There are three important system calls involved in setting up and using epoll:

1. **epoll\_create1()** → This API creates an **epoll instance** and returns a file descriptor representing it. We can think of this as the manager creating the central system through which all the table bells will be monitored. However, simply creating the epoll instance does not yet tell the kernel which sockets we are interested in. For that, we have **epoll\_ctl()**.
2. **epoll\_ctl()** → This API is used to **add, modify, or remove file descriptors** from the epoll instance. The server tells the kernel which FDs it wants to monitor and what events it is interested in, such as whether a socket is ready for reading.

In our restaurant analogy, this is like installing a bell on a particular table and telling the manager:

*“Keep an eye on this table and let me know when it needs attention.”*

For example, our server first registers the **listening socket** with `epoll_ctl()`. If a new client connection is waiting, that listening socket becomes ready, and `epoll` can notify the server. Then kernel can wake up the server and then once server accepts that, then this client fd also gets registered in kernel until the client is done with the request and response.

3. **`epoll_wait()`** → Once the `epoll` instance has been created and the required FDs have been registered, the server calls `epoll_wait()`.

This system call puts the server thread into a **waiting state** until one or more registered file descriptors become ready for an I/O event, or until another specified condition occurs. When an event occurs, the kernel wakes the server and `epoll_wait()` returns the **number of ready events**. The server can then examine those events and determine which file descriptors require attention.

**For example:** if a new client tries to connect, the listening socket becomes ready. `epoll_wait()` returns an event for that listening socket, and the server can then call `accept()` to accept the new connection. Similarly, if an already-connected client sends data, its socket becomes ready for reading. The server is notified through `epoll` and can then perform `recv()` on that socket. So in this case, kernel will have the fd of the client and hence by that server know that it is already connected and only needs to send data.

Hence, that's the basic idea of how `epoll` works:

**Create the `epoll` instance → Register the FDs → Wait for ready events → Handle only the FDs that need attention.**

#### Δ How does `epoll` handle events?

When the server accepts a new client connection, it registers the file descriptor (FD) of the client socket with the `epoll` instance. From this point onwards, the kernel monitors this FD for the events that the server is interested in. So let's discuss this in a little more detail:

1. **Connection Request:** When a new client tries to connect, the listening socket becomes ready, and `epoll` notifies the server. The server then calls `accept()` to establish the TCP connection. Once the connection is established, the newly created client FD is registered with `epoll` so that the server can be notified when the client socket becomes ready for I/O.
2. **I/O Request:** Once the client sends data, the corresponding client socket becomes ready for reading. `Epoll` identifies the fd of this client (as registered in case 1) and hence notifies the server, which can then receive the data and send the echoed data back to the client. The server has to receive and send data back.

This receiving and sending data back becomes **`epoll` events**. The two events relevant to our implementation are:

1. **EPOLLIN:** This tells the server that this client FD is ready for the server to **READ** from it. Hence if we use **EPOLLIN**, this just means that server is ready to receive the request and then can process and send it back. The server can therefore call `recv()` to receive the client's data.

2. **EPOLLOUT:** This tells the server that the client FD is ready for the server to **write** to it. In our implementation, if a previous send() could not send all the data, the remaining data is stored in the client's output buffer. EPOLLOUT allows the server to send this remaining data when the socket becomes writable otherwise we might have partial writing.

Now, to understand EPOLLOUT, let's take a small example: Suppose kernel has a send buffer of just 1 KB but the server received the request of 2KB. Now, at a moment, server could only send 1KB of data and then goes to either waiting state or next client. This means the rest of the 1KB just got lost.

For example:

Client messaged: Hey There, Mister

Echo server messaged: Hey There, Mist

Suppose till here whole kernel send buffer got filled. Due to which the client received only a part of data. This is called **Partial Writing**.

And to avoid this partial writing, we have EPOLLOUT, i.e., we maintain a client state, storing a part of his data in the **per - client output buffer**. When the socket becomes writable again, EPOLLOUT notifies the server, and the remaining data is sent.

Thus, the combination of **output buffering and EPOLLOUT** allows our server to correctly handle partial writes and ensure that the complete echoed data is eventually sent to the client.

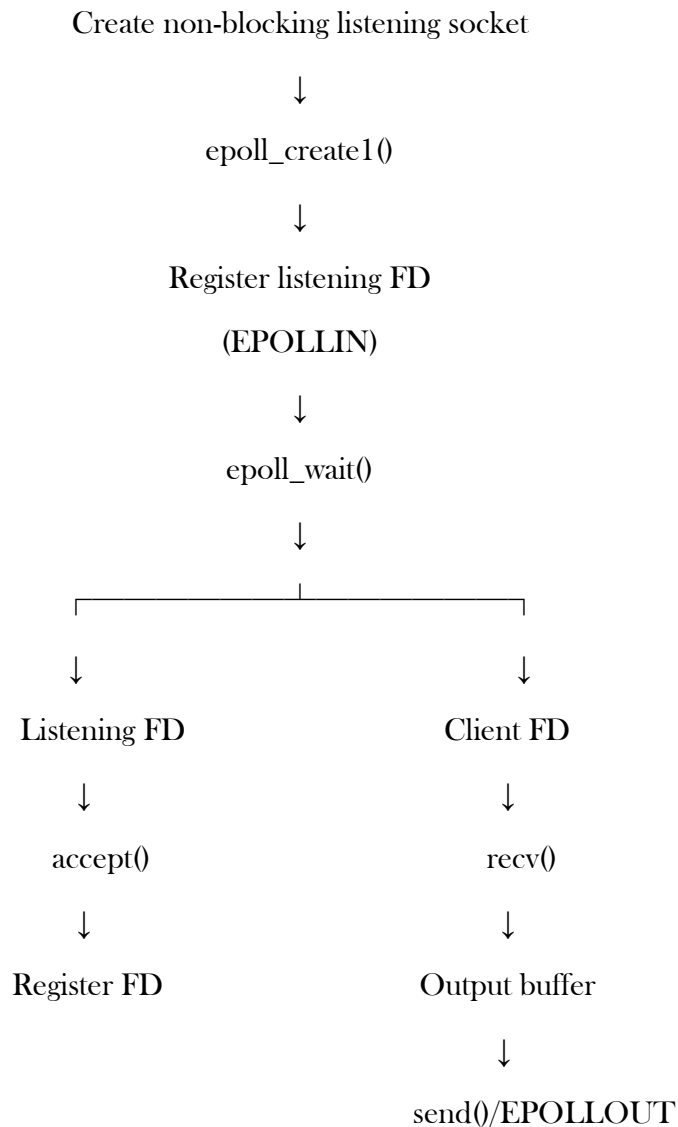
#### Δ Our epoll implementation:

So, let's quickly discuss how we implemented our epoll:

1. So, the very basic step we started with is to create a TCP socket by creating, binding to IP Address and port, and then set it to listen for incoming connections; making it a **listening socket**. Then we can make our socket non blocking using fcntl() and O\_NONBLOCK.
2. Once our socket is created, then we can introduce our **Epoll** instance using the epoll\_create1(). And then we can connect our epoll to the server.
3. Once we have joined/registered our epoll to the server, then our server and epoll is ready to work.
4. So, we can let our server go to waiting state and let kernel wake our server up if any client comes.
5. Now, we can have two forms of requests:
  - **Connection request:** So our listening socket server can accept the connection and hence then once connection is established then we can register client fd and give to kernel so that kernel can wake us up if this client becomes ready and try to send data.
  - **I/O Request:** Now in this case, the connected client tries to send the data to the connection socket. So firstly, we can check if we have some unsent data to be sent or not using EPOLLOUT that tells it is ready to write. Once that is flushed, then we can receive the data from the client and then can process and send it back. Here's where EPOLLIN will come to allow us to retrieve the data.

Hence, that's the overview of how the epoll works.

A rough diagram showing this high level view is given below:



→ **io\_uring Multiplexing:**

Δ **What is io\_uring?**

Introduced in Linux (2019) to replace legacy, readiness-based notification mechanisms like epoll, io\_uring is a high-performance asynchronous I/O framework built on two lockless circular ring buffers—a Submission Queue (SQ) and a Completion Queue (CQ)—shared directly between user space and kernel space via memory mapping (mmap).

By shifting from readiness notifications to an asynchronous completion model, io\_uring enables applications to submit batched I/O operations and harvest results without metadata copying or frequent system calls, eliminating costly CPU context switches and significantly boosting throughput in high-concurrency systems.

Δ **How does io\_uring work?**

→ **Ring Buffer Architecture & Shared Memory (mmap):**

The core innovation of `io_uring` is using shared memory between user space and kernel space to eliminate memory copies during I/O submission and completion.

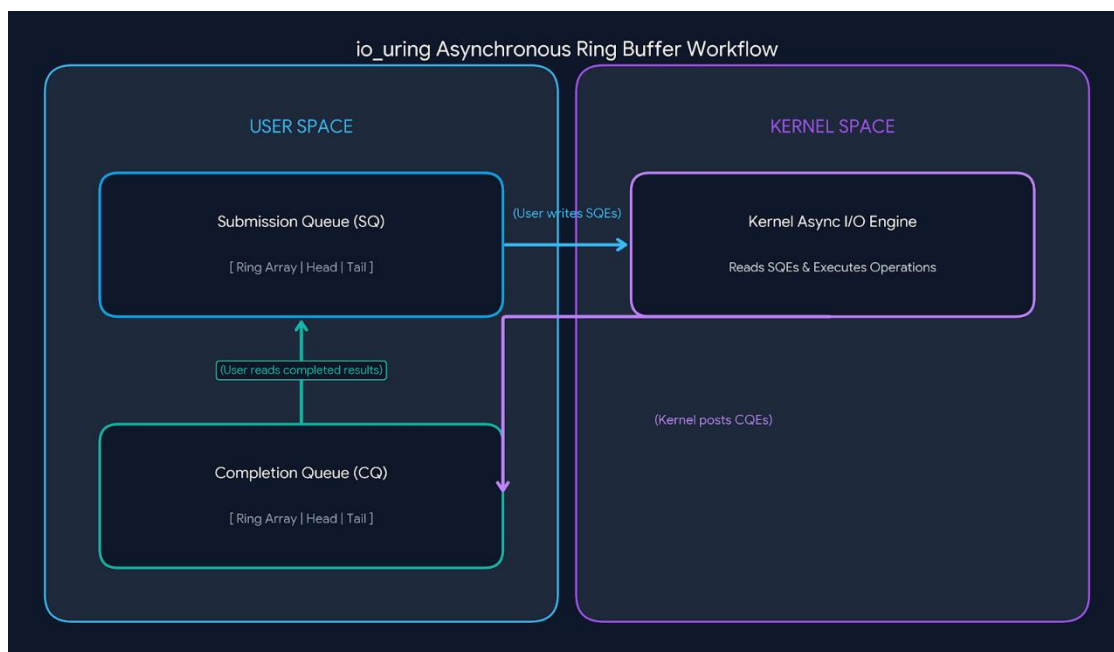
When an application initializes `io_uring`, the kernel creates two lockless circular ring buffers in kernel memory:

1. **Submission Queue (SQ):** Used by user space to submit asynchronous I/O requests to the kernel.
2. **Completion Queue (CQ):** Used by the kernel to return operation results back to user space.

The application calls `mmap()` to map these kernel rings directly into its address space.

#### → Memory Mapping Steps

1. **SQ Ring (`IORING_OFF_SQ_RING`):** Contains control pointers (`head`, `tail`, `ring_mask`) and an index array mapping queue positions to SQE structures.
2. **SQE Array (`IORING_OFF_SQES`):** A flat array of `struct io_uring_sqe` elements holding request metadata.
3. **CQ Ring (`IORING_OFF_CQ_RING`):** Contains control pointers (`head`, `tail`, `ring_mask`) and an array of `struct io_uring_cqe` elements holding operation outcomes.



#### → Data Structures: SQE and CQE

Submission Queue Entry (`struct io_uring_sqe`):

```

struct io_uring_sqe {
    __u8  opcode;      // Operation type (e.g., IORING_OP_ACCEPT, IORING_OP_RECV)
    __u8  flags;        // SQE flags (e.g., IOSQE_IO_LINK for chained operations)
    __u16 ioprio;       // I/O priority
    __s32 fd;          // File descriptor (socket fd, server listen fd)
    __u64 off;         // Offset for file I/O
    __u64 addr;        // Pointer to user buffer (e.g., buffer address for recv/send)
    __u32 len;         // Buffer length or vector count
    __u32 accept_flags; // Flags for accept operations (e.g., SOCK_NONBLOCK)
    __u64 user_data;    // 64-bit user tag passed unchanged to the CQE
    /* ... additional internal flags and padding ... */
};

```

Completion Queue Entry (struct io\_uring\_cqe):

```

struct io_uring_cqe {
    __u64 user_data;    // Matches the user_data field set in the original SQE
    __s32 res;          // Operation result (equivalent to syscall return: bytes transferred or -errno)
    __u32 flags;        // CQE flags (e.g., buffer flags)
};

```

## → System Calls & Kernel API

io\_uring\_setup()

```

int io_uring_setup(unsigned entries, struct io_uring_params *p);

```

Creates an io\_uring instance with an SQ/CQ depth defined by entries (must be a power of 2). The kernel fills struct io\_uring\_params with offset information needed for subsequent mmap() calls and returns a ring file descriptor (ring\_fd).

io\_uring\_register()

```

int io_uring_register(int ring_fd, unsigned opcode, const void *arg, unsigned nr_args);

```

Registers kernel resources (e.g., pre-mapped user buffers or file descriptor tables) to optimize execution paths further.

```

int io_uring_enter(int ring_fd, unsigned to_submit, unsigned min_complete, unsigned flags);

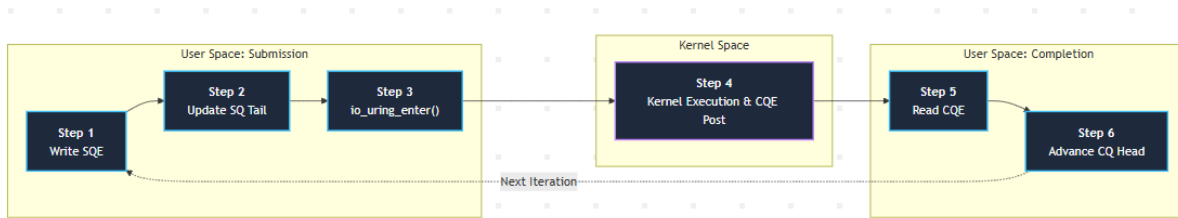
```

Informs the kernel that new SQEs are pending in the SQ ring buffer, or blocks user space until min\_complete events are posted in the CQ ring.

- **to\_submit:** Number of SQEs submitted in this call.
- **min\_complete:** Minimum number of completed CQEs required before returning.

- **flags:** Behavior modifiers (e.g., `IORING_ENTER_GETEVENTS`).

## → Life Cycle:



**Step 1: Write SQE (User Space)** – App populates an available SQ slot with request metadata (opcode, file descriptor, buffer address, user\_data).

**Step 2: Update SQ Tail (User Space)** – App advances sq\_tail atomically to publish new SQEs into shared memory.

**Step 3: Execute io\_uring\_enter() (User → Kernel)** – App calls io\_uring\_enter() to notify kernel background workers of pending SQEs.

**Step 4: Execute & Post CQE (Kernel Space)** – Kernel processes I/O asynchronously, populates a CQE with the return result, and updates cq\_tail.

**Step 5: Read CQE (Kernel → User)** – App retrieves completed CQEs directly from shared memory using cq\_head.

**Step 6: Advance CQ Head (Loop Back)** – App advances cq\_head to release processed CQ slots and repeats the event loop cycle.

## → Lockless Memory Ordering & Synchronization

The SQ and CQ rings operate without locks because they follow a Single-Producer / Single-Consumer (SPSC) model:

- **Submission Queue:**
  - **Producer:** User space (writes entries and updates \*sq\_tail).
  - **Consumer:** Kernel space (reads entries and updates \*sq\_head).
- **Completion Queue:**
  - **Producer:** Kernel space (writes entries and updates \*cq\_tail).
  - **Consumer:** User space (reads entries and updates \*cq\_head).

Synchronization across CPU cores is maintained using atomic operations and memory barriers (smp\_store\_release and smp\_load\_acquire), ensuring lock-free concurrency between user space and kernel space.

## → The Analogy of a Restaurant:

Imagine a busy restaurant that replaces traditional waiters with two circular conveyor belts running between the dining room (User Space) and the kitchen (Kernel Space):

- **The Submission Queue (SQ) Belt – Placing Orders:** When customers want to order food, drink, or pay, they write their request on a standardized ticket and place it on the incoming conveyor belt. The kitchen picks up these tickets automatically. Customers don't need to call a waiter to take the order to the kitchen.
- **Kernel Processing – Background Cooking:** The chef (Kernel) works asynchronously on the tickets coming off the submission belt.
- **The Completion Queue (CQ) Belt – Serving Finished Food:** As soon as a dish is ready, the kitchen places the completed meal along with its original ticket number onto the return belt. The restaurant staff simply watches the return belt and picks up completed dishes in batches to hand to customers.

→ Need of io\_uring?

### 1. Eliminating Per-Operation System Call Overhead

- **What It Solves:** In `epoll`, handling  $N$  active connections requires 1 `epoll_wait` call followed by  $N$  individual system calls (`recv()` or `send()`). At high packet rates, context switches dominate CPU usage.
- **How io\_uring Solves :** It operates on an asynchronous completion model. Applications queue multiple requests into a Submission Queue (SQ) and notify the kernel using a single batched `io_uring_enter()` call (or zero system calls when using kernel polling mode `SQPOLL`).

### 2. Eliminating User-Kernel Memory Copying

- **What It Solves:** Transferring request metadata and file descriptor sets back and forth between user space and kernel space consumes memory bandwidth and CPU cycles.
- **How io\_uring Solves It:** It utilizes two lockless circular ring buffers—the Submission Queue (SQ) and Completion Queue (CQ)—mapped directly into memory shared between user space and kernel space (`mmap`). User space writes Submission Queue Entries (SQEs) and reads Completion Queue Entries (CQEs) without copying memory across kernel boundaries.

### 3. Removing Post-Accept Configuration Round-Trips

- **What It Solves:** Standard event loops require additional system call round-trips after `accept()` (e.g., executing `fcntl(fd, F_SETFL, O_NONBLOCK)`) to configure newly created client sockets.
- **How io\_uring Solves It:** It allows socket configuration flags to be specified directly inside the initial request. For instance, passing `SOCK_NONBLOCK` inside an `IORING_OP_ACCEPT` SQE forces the kernel to return an already non-blocking descriptor upon completion.

### 4. Unifying Network and Disk I/O Interfaces

- **What It Solves:** Linux previously lacked a single asynchronous framework. Network I/O relied on `epoll`, while file system I/O relied on Linux AIO (`io_submit`), which often blocked unexpectedly on file metadata operations.
- **How io\_uring Solves It:** It provides a single kernel execution framework that handles network sockets, block storage files, and system timers using identical Submission Queue Entry (SQE) opcodes (`IORING_OP_ACCEPT`, `IORING_OP_RECV`, `IORING_OP_READV`, `IORING_OP_WRITEV`).

## 🌀 Benchmarking & Performance Analysis:

### → Performance Data:

| Server  | N     | Bandwidth<br>(Mbps) | Latency p99 (ms) | Avg CPU (%) | Avg Vol CS | Avg Invol CS | Total<br>Syscalls |
|---------|-------|---------------------|------------------|-------------|------------|--------------|-------------------|
| select  | 10    | 4465.43             | 0.6              | 0.00        | 0.00       | 10.00        | 22219             |
| select  | 100   | 4343.13             | 0.8              | 0.00        | 0.11       | 10.00        | 54181             |
| select  | 1000  | 4024.57             | 0.7              | 0.20        | 9.40       | 0.20         | 56419             |
| select  | 5000  | NaN                 | NaN              | 36.10       | 895.50     | 32.00        | 142053            |
| select  | 10000 | NaN                 | NaN              | 36.20       | 897.40     | 32.80        | 194756            |
| poll    | 10    | 4300.52             | 0.2              | 0.00        | 0.00       | 10.00        | 22624             |
| poll    | 100   | 4278.13             | 0.2              | 0.00        | 0.00       | 10.00        | 54376             |
| poll    | 1000  | 4050.45             | 0.6              | 0.10        | 9.70       | 0.20         | 56600             |
| poll    | 5000  | 2363.99             | 81.2             | 2.87        | 33.98      | 0.30         | 90943             |
| poll    | 10000 | 2340.36             | 2614.3           | 45.12       | 28.83      | 235.02       | 120605            |
| epoll   | 10    | 3239.44             | 0.2              | 0.00        | 0.00       | 10.00        | 22301             |
| epoll   | 100   | 3959.91             | 0.2              | 0.00        | 0.00       | 10.00        | 53943             |
| epoll   | 1000  | 3468.95             | 0.4              | 0.10        | 14.70      | 0.10         | 56849             |
| epoll   | 5000  | 2746.88             | 91.3             | 0.30        | 76.00      | 0.20         | 102307            |
| epoll   | 10000 | 2500.64             | 2567.9           | 1.29        | 621.57     | 5.78         | 140623            |
| iouring | 10    | 4417.37             | 0.2              | 0.00        | 0.00       | 10.00        | 6974              |
| iouring | 100   | 4311.69             | 0.2              | 0.00        | 0.00       | 10.00        | 33814             |

|         |       |         |       |      |        |        |       |
|---------|-------|---------|-------|------|--------|--------|-------|
| iouring | 1000  | 3109.16 | 0.5   | 0.10 | 12.40  | 0.10   | 29695 |
| iouring | 5000  | 2756.14 | 52.3  | 0.40 | 71.59  | 0.20   | 8404  |
| iouring | 10000 | 2472.42 | 255.7 | 2.27 | 885.53 | 238.85 | 10433 |

## → Project Overview & Methodology

We evaluated the performance and scalability of four distinct Linux socket-handling paradigms by implementing single-threaded TCP echo servers in C/C++. The study swept across five concurrency levels ( $N \in \{10, 100, 1000, 5000, 10000\}$ ) to observe the transition from low-load networking to high-concurrency CPU bottlenecking.

### Testing Architecture:

- **Target Environment:** Ubuntu Linux (Kernel 7.0.0-31), executed on an ARM64 Apple Silicon virtual machine.
- **Load Generation:** tcphkali was utilized to generate concurrent connections, with uncapped message rates for throughput tests and a capped rate of 100 msg/s for latency profiling.
- **Profiling Tools:** CPU utilization and context switches were sampled at 1-second intervals using pidstat, while system call overhead was traced using strace. Data was extracted using Bash, awk, and regex into structured CSV formats.

## → Theoretical Motivation

The core objective was to empirically document the evolution of Linux network stack efficiency:

- **1. The  $O(N)$  Paradigm:** Mechanisms like select and poll require the kernel to iterate over all managed file descriptors linearly on every event loop iteration, introducing immense memory copy and CPU cache thrashing at high connection loads.
- **2. The  $O(1)$  Paradigm:** epoll solves readiness efficiency by using a kernel-managed Red-Black tree and returning only active events to user space.
- **3. The Asynchronous Paradigm:** While epoll solves readiness, actual I/O still requires heavy context switching (read()/write()). io\_uring utilizes shared-memory ring buffers (Submission Queue and Completion Queue) to decouple I/O initiation from execution, virtually eliminating the system call tax.

## → The Approach to "Fair" Benchmarking

To ensure our comparisons were scientifically valid, we enforced strict fairness parameters across all four engines:

- **Standardized Buffer Sizes:** All engines utilized a consistent 64 KB output buffer capacity per client (OUT\_BUF\_CAP = 65536) to normalize memory footprint expectations.
- **Shared Network Utilities:** All engines relied on a common network\_utils.c module for socket creation and TCP tuning, ensuring identical initial states (e.g., applying TCP\_NODELAY).

- **Artificial Caps:** We explicitly capped poll and io\_uring at a MAX\_CLIENTS threshold of 10,000. This prevented unbounded engines from consuming disproportionate system resources during the final testing sweep, allowing for an apples-to-apples comparison against epoll.

## → Discovery & Discrepancies

While our data validated several core networking theories, our most valuable insights came from tracing empirical anomalies directly back to our C implementations.

### 1. The printf Head-of-Line Bottleneck in epoll

- **The Issue:** Despite its  $O(1)$  efficiency, our epoll server exhibited a massive 99th-percentile latency spike (2,567.9 ms) at  $N=10,000$ , nearly identical to the inefficient poll engine.
- **The Cause:** We embedded a synchronous `printf("New client connected...")` statement within the `while (1) { accept() } queue-drain loop`. When 10,000 connections were instantiated simultaneously by tcpkali, the single thread blocked for seconds writing strings to standard output, completely starving existing clients of processing cycles.

### 2. The select Ghost Load Spin

- **The Issue:** At  $N=10,000$ , the select server transferred 0 Mbps of data, yet consumed 36.2% of the CPU and generated 194,756 system calls.
- **The Cause:** select is strictly limited by the POSIX `FD_SETSIZE` macro of 1,024. Our code gracefully handled this by printing a rejection message and closing connections exceeding this limit. However, at 10,000 active connection attempts, the thread became trapped in a relentless `accept() -> printf() -> close()` loop, generating ~95,000 accept/close syscall pairs over 10 seconds without ever transferring network payloads.

### 3. The Unfair fcntl System Call Tax

- **The Issue:** We inadvertently handicapped the poll and epoll engines during connection setup.
- **The Cause:** Our shared `network_utils.c` toggled sockets to non-blocking using `fcntl(F_GETFL)` and `fcntl(F_SETFL)`. At  $N=10,000$ , this resulted in 20,004 unnecessary context switches for both epoll and poll. Conversely, our io\_uring implementation idiomatically set `sqe->accept_flags = SOCK_NONBLOCK` directly inside the Submission Queue Entry. This completely bypassed the fcntl syscalls, providing io\_uring with an architectural advantage in the benchmark that went beyond standard I/O multiplexing.

### 4. The O(K) Memory Shifting Penalty

- **The Issue:** In our `flush_outbound_buffer` logic across all synchronous engines, partial `send()` completions (e.g., hitting EAGAIN) triggered a memmove operation.
- **The Cause:** Under heavy network saturation, our single thread was physically shifting up to 64 KB of memory per socket down the array on every partial send. This destroyed CPU caches and introduced an artificial compute bottleneck. A zero-copy ring buffer utilizing read/write pointer offsets would have yielded substantially higher throughput.

## → Benchmark Analysis

### Δ Abstract

The empirical results powerfully validated core operating systems theory: the shift from  $O(N)$  linear scanning to  $O(1)$  readiness drastically reduced CPU thrashing. Furthermore, shifting from a readiness model to `io_uring`'s asynchronous completion model reduced system call boundary crossings by over 92% under maximum load.

However, the benchmark also proved that kernel-level efficiency can be completely undermined by application-layer artifacts. We discovered two significant implementation bottlenecks that artificially shaped our data: a  $O(N)$  terminal logging block that destroyed `epoll` tail latencies, and a TCP rejection loop in `select` that caused extreme system call spikes.

### Δ The Approach to Fair Benchmarking

To ensure a scientific comparison, all engines shared identical memory constraints (64 KB output buffers per client) and utilized a common `network_utils.c` networking stack. We placed a hard cap of 10,000 max clients on the unbounded engines (`poll`, `epoll`, `io_uring`) so they could be directly compared to each other at the absolute maximum concurrency tier without unbounded growth skewing memory footprints.

### Δ Implementation Bottlenecks (Mistakes & Discoveries)

The data yielded critical insights into how implementation flaws shape performance graphs:

- **1. The `printf` Head-of-Line Block:**
  - **The Issue:** Despite its  $O(1)$  efficiency, our `epoll` server exhibited a massive 99th-percentile latency spike (2,567.9 ms) at  $N=10,000$ , nearly identical to the inefficient `poll` engine.
  - **The Cause:** We embedded a synchronous `printf("New client connected...")` statement within the `while (1) { accept() } queue-drain loop`. When 10,000 connections were instantiated simultaneously by `tcpkali`, the single thread blocked for seconds writing strings to standard output, completely starving existing clients of processing cycles.
- **2. The `select` Ghost Load Spin:**
  - **The Issue:** At  $N=10,000$ , the `select` server transferred 0 Mbps of data, yet consumed 36.2% of the CPU and generated 194,756 system calls.
  - **The Cause:** `select` is strictly limited by the `POSIX FD_SETSIZE` macro of 1,024. Our code gracefully handled this by printing a rejection message and closing connections exceeding this limit. However, at 10,000 active connection attempts, the thread became trapped in a relentless `accept() -> printf() -> close()` loop, generating ~95,000 `accept/close` syscall pairs over 10 seconds without ever transferring network payloads.
- **3. The Unfair `fcntl` System Call Tax:**
  - **The Issue:** We inadvertently handicapped the `poll` and `epoll` engines during connection setup.

- **The Cause:** Our shared network\_utils.c toggled sockets to non-blocking using `fcntl(F_GETFL)` and `fcntl(F_SETFL)`. At  $N=10,000$ , this resulted in 20,004 unnecessary context switches for both `epoll` and `poll`. Conversely, our `io_uring` implementation idiomatically set `sqe->accept_flags = SOCK_NONBLOCK` directly inside the Submission Queue Entry. This completely bypassed the `fcntl` syscalls, providing `io_uring` with an architectural advantage in the benchmark that went beyond standard I/O multiplexing.

#### • 4. The O(K) Memory Shifting Penalty:

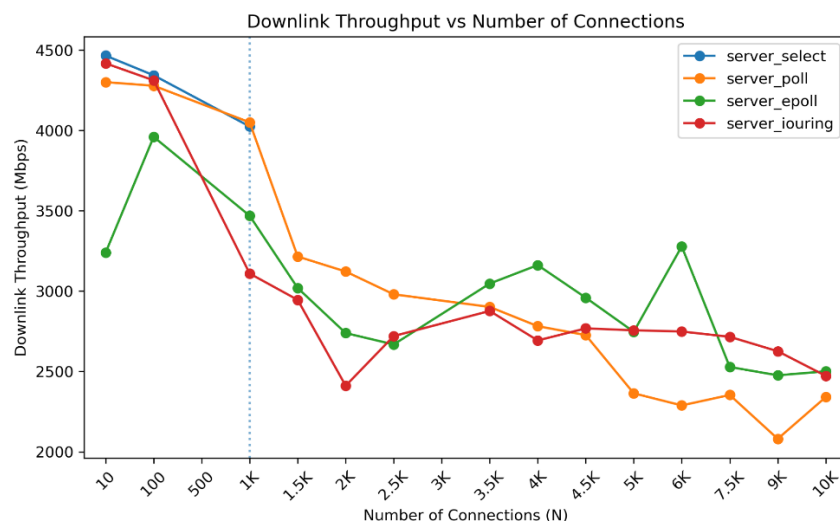
- **The Issue:** In our `flush_outbound_buffer` logic across all synchronous engines, partial send() completions (e.g., hitting `EAGAIN`) triggered a `memmove` operation.
- **The Cause:** Under heavy network saturation, our single thread was physically shifting up to 64 KB of memory per socket down the array on every partial send. This destroyed CPU caches and introduced an artificial compute bottleneck. A zero-copy ring buffer utilizing read/write pointer offsets would have yielded substantially higher throughput.

### → Graph-by-Graph Analysis

Below is the theoretical expectation and empirical reality for all seven performance metrics tracked during the concurrency sweep.

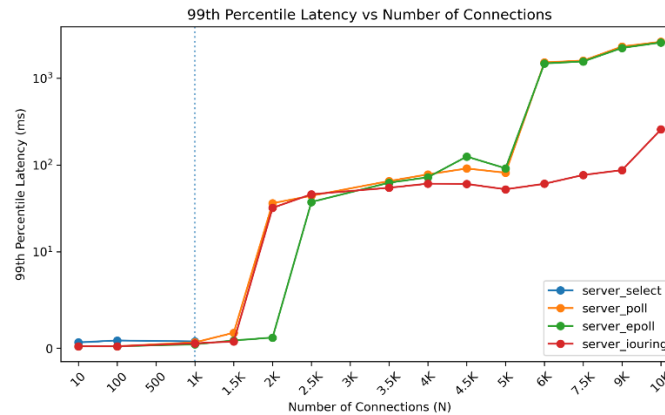
#### 1. Downlink Throughput (Bandwidth Mbps vs. Concurrency)

- **The Baseline Expectation:** All engines should peak uniformly at low connections, dropping slightly as the kernel spends more time managing state. `select` should flatline completely at its hard limit.
- **Empirical Observation:** At 10 connections, all servers performed optimally, moving  $\sim 4,400$  Mbps. By 10,000 connections, multiplexing overhead dragged `poll`, `epoll`, and `io_uring` down to roughly 2,300 - 2,500 Mbps.
- **The select Cliff:** `server_select` drops abruptly from 4,024 Mbps at  $N=1000$  to exactly NaN (0 Mbps) at  $N=1500$  and beyond. This visualizes the hard POSIX limitation where `select()` physically cannot map a file descriptor integer greater than 1024 into its bitmask.



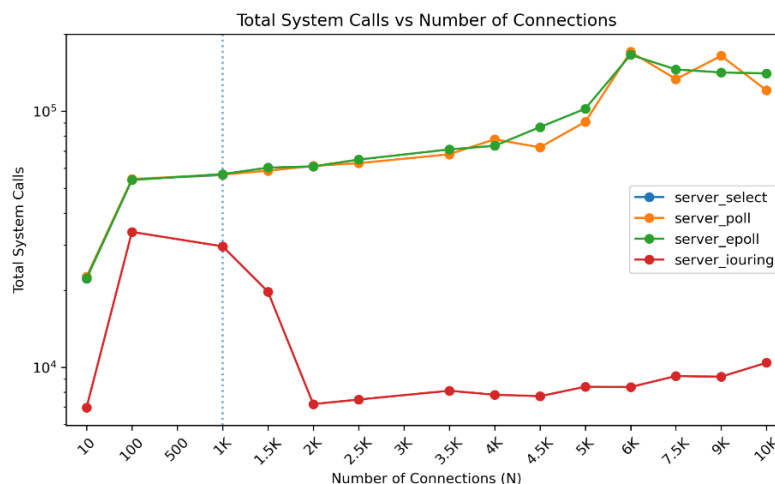
## 2. 99th Percentile Latency (p99 ms vs. Concurrency)

- **The Baseline Expectation:** The  $O(N)$  architecture of poll should cause exponential latency growth (Head-of-Line blocking) while epoll and io\_uring should remain flat.
- **Empirical Observation:** poll behaved exactly as expected, ballooning from 0.6 ms at  $N=1000$  to a massive 2,614.3 ms at  $N=10,000$ . io\_uring successfully decoupled processing, maintaining an excellent tail latency of just 255.7 ms.
- **The epoll Anomaly:** Theoretically, epoll should have matched io\_uring. Instead, it spiked to 2,567.9 ms. As identified in our code audit, this was an application-layer failure caused by a synchronous `printf()` inside the `accept()` loop blocking the single thread during the 10,000 conn/s connection burst.



## 3. Total System Calls (Syscalls vs. Engine)

- **The Baseline Expectation:** Legacy synchronous engines require a system call for readiness (epoll\_wait/poll) and separate calls for I/O (read/write). io\_uring batches these, predicting a drastic drop.
- **Empirical Observation:** This graph proves the true power of asynchronous networking. At 10,000 connections, epoll generated 140,623 system calls. io\_uring processed the same load with only 10,433 system calls. By replacing pairs of readiness/execution syscalls with a single io\_uring\_enter, the engine yielded a spectacular  $\approx 92\%$  reduction in user-to-kernel boundary crossings.



## 🌀 Conclusion:

This project successfully maps the historical and technical evolution of Linux network I/O, demonstrating the necessary transition from the synchronous,  $O(N)$  limitations of select and poll to the event-driven,  $O(1)$  efficiency of epoll, and ultimately to the lockless, asynchronous completion model of io\_uring.

Our benchmarking data directly reinforced theoretical expectations. While select buckled under its hardcoded file descriptor limit and poll suffered from quadratic latency degradation due to continuous kernel-to-user array copies, epoll effectively managed high connection counts by acting only on active readiness events. Furthermore, io\_uring proved to be a generational leap in networking performance. By utilizing shared memory ring buffers, it successfully decoupled I/O submission from execution, batching operations to reduce system call overhead by approximately 92% under maximum load compared to epoll.

Equally important to the kernel-level metrics were the application-layer discoveries. The benchmarking anomalies—such as the epoll tail-latency spike caused by synchronous printf statements and the select engine's ghost-load CPU spin—highlight a critical engineering lesson: high-performance kernel multiplexing can be entirely undermined by inefficient user-space code. Ultimately, this project demonstrates that while io\_uring provides superior raw architectural capability for scalable I/O, realizing its full potential requires meticulous, zero-copy, and strictly non-blocking application design.

---

†††††